*AI Engineering and Claude Architect · Part IV, Production*

# AI Production Engineering, Lesson 4

## Observability: Traces, Tokens and Cost

The dashboard said latency was fine and errors were low. One trace showed a 30-second tool timeout, a cache miss costing two-thirds of the run, and the same call made twice. Agents need traces, not only metrics.

| | |
|---|---|
| Module | AI10 of 13 |
| Lesson | 4 of 7 |
| Prerequisites | AI10 L1–L3, AI06 L2, HLD M13 (observability) |
| Exam relevance | Tracing tools aren't in the guide. Supports TS 5.1, 5.3 |
| Reading time | ~30 minutes |

# Contents

1. Observability for AI Systems

*Lesson 4 of 7 · AI Production Engineering*

# Observability for AI Systems

### Key idea

**An HTTP 200 tells you nothing about whether the answer was right, so AI systems need three more signals: traces of each step, tokens and cost per outcome, and quality.** Read token fields carefully, as they're easy to double-count. Keep every error and slow trace by sampling after the outcome is known. Treat stored prompts as sensitive data.

> *Before: the agent loop, evals, and HLD's logs, metrics and traces, which aren't retaught here. Teaches: what to trace, reading a trace, token accounting, OpenTelemetry for Claude, sampling, percentiles, and drift. Used by: L5, L6, L7.*

## The hook: one trace

The dashboard is green. A customer complains that an agent took 40 seconds. Here's that run's trace, as invented data, costed at Sonnet 5.5 prices. Real output from `OpsLab.java`:

```
One agent run:
  agent.run              41200 ms  in      0  read      0  out    0  $0.0000
    model turn 1          2100 ms  in   1200  read   8000  out   90  $0.0041  cache hit
    tool search_orders     180 ms  in      0  read      0  out    0  $0.0000
    model turn 2          2400 ms  in    900  read   8000  out  120  $0.0038  cache hit
    tool get_invoice     30000 ms  in      0  read      0  out    0  $0.0000  TIMEOUT, retried
    tool get_invoice       240 ms  in      0  read      0  out    0  $0.0000
    model turn 3          3300 ms  in  14000  read      0  out  200  $0.0300  cache MISS, prefix changed
    tool get_invoice       220 ms  in      0  read      0  out    0  $0.0000  same call repeated
    model turn 4          2760 ms  in   1100  read  14000  out  310  $0.0067  cache hit
  total cost $0.0446
```

Four findings, and none of them would show on a request-level dashboard:

| Finding | Evidence | What it points to |
|---|---|---|
| The delay is in a tool, not the model | 30,000 of 41,200 ms | Fix `get_invoice`'s timeout |
| One turn is 67% of the cost | Turn 3's cache miss | Something changed the prefix |
| A call was repeated | Same tool, after success | The result may be getting trimmed, or the tool description is unclear |
| The run succeeded | No error anywhere | **Errors alone wouldn't have flagged it** |

**In an agent, the interesting failures return a 200.**

## Intuition: the flight recorder

An airline doesn't only count how many planes landed. Each plane carries a recorder, so after any odd flight you can replay every control input and reading. The tally says something's wrong, and the recorder says what.

Metrics are the tally, and traces are the recorder. You need both, and you need to choose what to record, because a recorder can also store things that passengers wouldn't want kept.

## Mechanism

### The signals

HLD's logs, metrics and traces all still apply. Here's what's added:

| Signal | Examples | Answers |
|---|---|---|
| **Latency** | Time to first token, total, by model and step | Is it slow, and where? |
| **Tokens** | Input, cache write, cache read, output | Why does it cost this? |
| **Cost** | Per request, feature, tenant, **successful outcome** | Is it worth it? |
| **Cache** | Hit ratio, by route | Did a release break the prefix? |
| **Outcome** | Stop reasons, refusals, truncation, tool errors, steps | Did it finish properly? |
| **Quality** | Sampled evals, user feedback, escalations | Was it right? |
| **Capacity** | 429s, 529s, remaining-quota headers | How close to the limit? |

**Time to first token and total time are separate metrics.** Chat users feel the first, and batch jobs the second. A single "latency" chart mixes them.

**Watch stop reasons.** A rise in `max_tokens` stops means truncated answers. A rise in `refusal` may mean a change in traffic or an abuse attempt. Both return success at HTTP level.

## What a trace should hold

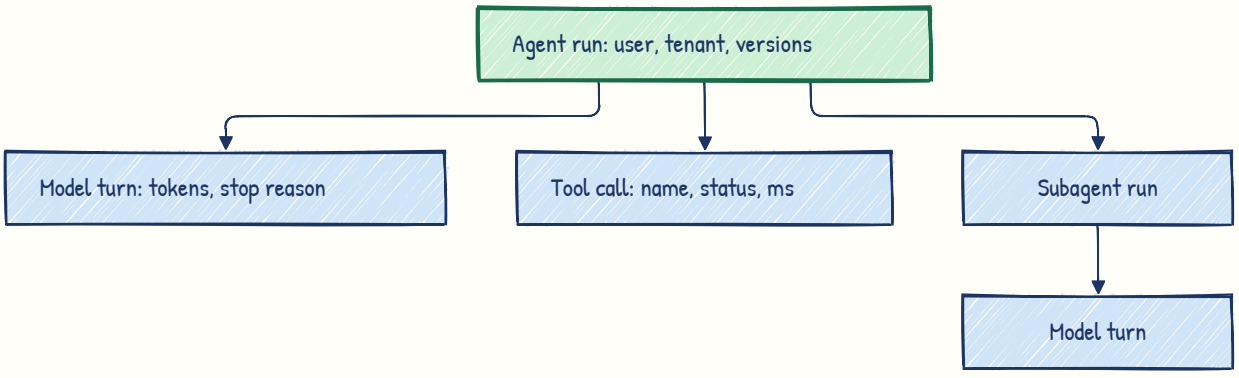



*Figure 1. One trace per agent run. Each model turn and tool call is a span, so time and cost add up across the tree.*

Record these on the run: prompt version, model ID, tool versions, tenant and feature, and the eval or canary group. AI03 L2 raised version logging. Without it, you can't link a bad output to the change that caused it. Also keep the API's `request-id`, for support.

## Token accounting: the most common error

Three sets of rules meet here, and they don't agree.

**The Messages API.** `input_tokens` covers only tokens **after the last cache breakpoint**. From the rate limits page:

`total_input_tokens = cache_read_input_tokens + cache_creation_input_tokens + input_tokens`

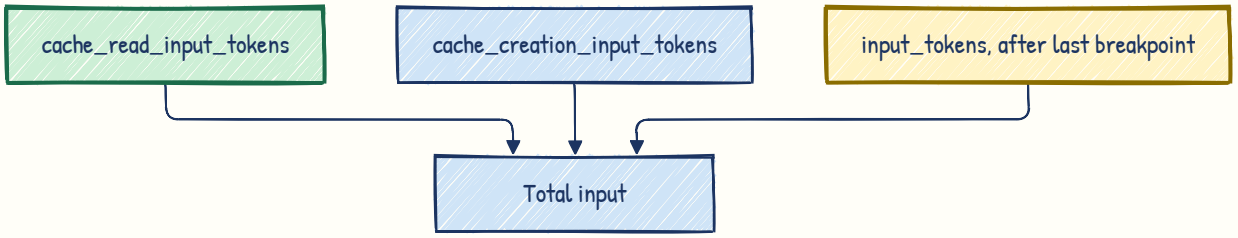



*Figure 2. Three API fields add up to the true input. Each has its own price, and only two of them count toward the rate limit on most models.*

Real output, using the docs' own example of a large cached document with a short question:

```
Reading usage correctly. API response:
  input_tokens=50, cache_read_input_tokens=200000, output=400
  total input: 200,050
  'cache hit ratio' using input only: 400000.0%, which is impossible
  correct: 99.98%
  real cost on Sonnet 5.5: $0.0241
  cost if reads were billed as input: $0.4041
  cost if input_tokens alone were billed: $0.0041
```

A dashboard built on `input_tokens` alone is wrong by a factor of six on cost here, and it would look wonderful.

**OpenTelemetry spans.** According to the bundled API reference, `gen_ai.usage.input_tokens` **already includes** cache reads and writes, unlike the API response. So adding the cache attributes to it counts them twice. **The same name means different things in two places, so write down which one your dashboard sums.**

**More double-counting traps,** from the same reference:

| Trap | Rule |
| --- | --- |
| A request can have an SDK span and an API span | Sum one source per request. Prefer API spans when both exist |
| Usage repeats on parent and child levels | Sum only the named spans, such as `anthropic.messages.create` |
| Spans can arrive more than once | Deduplicate by trace ID plus span ID |
| A count from `count_tokens` | It's an estimate, not usage, so leave it out |
| Fallback served the request | The SDK span has usage for the model that served it |

Check your totals against the provider's numbers. The Usage and Cost Admin API reports usage and cost, and if your own dashboard disagrees with the invoice, your dashboard is wrong.

## OpenTelemetry for Claude

The API reference describes two independent sources. **It's in beta, and names and options can change,** so pin your SDK version.

| | SDK tracing | API tracing |
| --- | --- | --- |
| Emitted by | The SDK, in your process | The API, server-side |
| Set up by | Code | An admin adds a destination |
| Works on | Every cloud platform's client | Claude API and Claude Platform on AWS |
| Code change needed | Yes | No |

Together, the API's spans nest under the SDK's span of the same request. The `traceparent` header connects API spans to your own trace.

**Content is off by default.** The default mode is `metadata_only`: identifiers, timing, status and token counts. Full content mode adds prompts and responses. The reference warns that the backend then stores them under **its own** access and retention rules. A conversation ID is exported in both modes, so never put personal data in it.

The field's shared conventions for GenAI spans have moved to their own OpenTelemetry repository. Its README lists no stability level, and gives its schema URL as "TODO," so expect change. I read only the README, so I won't list attribute names from it here.

## Prompts in logs are personal data

AI09 L4 covered this as a data store. The observability version:

| Option | Debugging value | Risk |
|---|---|---|
| Metadata only | Timing and tokens | Low |
| Hash of prompt | Finds repeats, and prefix changes | Low |
| Redacted content | Moderate, with the detector's misses | Moderate |
| Sampled full content, restricted, short retention | High | Managed |
| All content, kept forever | High | **A breach waiting to happen** |

## Sampling: decide after the outcome

Storing every trace is costly. Real output, with invented rates of 0.2% errors and 1% slow:

```
Sampling:
  1,000,000 traces, 1,973 errors, 10,014 slow
  1% head sampling: keep 10,133 traces, including 21 errors
  tail sampling:    keep 12,903 traces, including 1,973 errors
```

Head sampling decides at the start, so it loses 99% of what you most want. Tail sampling decides at the end, and keeps every error for about the same storage. It needs buffering until a trace completes, usually in a collector. Long agent runs make that buffer larger.

Add random sampling on top of that, so that you also see normal behaviour, and sample by segment, so that small tenants aren't lost.

## Percentiles, not means

A mix of 90% chat and 10% agent runs, on invented latencies:

```
Latency mix of 90% chat and 10% agent runs:
  mean 4.9 s, p50 2.1 s, p95 25.1 s, p99 47.6 s
```

The mean describes nobody. Chat users see about 2 seconds, and agent users about 25. Split by route **before** taking percentiles, as a percentile across a mixture describes neither group well.

## Drift

Without any change on your side, behaviour can shift, because users' questions change. AI01 L4 introduced drift. A simple alarm compares the share of question types with a baseline, using the population stability index:

```
Drift in the share of question types:
  small change PSI 0.002
  new topic PSI 0.891
```

Thresholds around 0.1 and 0.25 are commonly quoted, and I haven't verified a source for those, so tune them yourself. A cheap classifier can provide the categories. A drift alarm means "go and read samples, and consider adding tasks to the suite." It doesn't tell you that something's wrong.

Other drift signals: input length, tool use per run, refusal rate, and retrieval scores (AI05 L6).

### Alerts

| Alert | Why it's worth a page |
|---|---|
| Spend cap or auth errors | Total outage, and retries won't help (L1) |
| Error rate or p95 by route | The usual |
| Spend per hour against budget | Runaway loops and abuse |
| Cache hit ratio falls after a deploy | Cost and quota problem (L1) |
| Max-steps or token-limit endings rise | Agents are looping |
| Deprecated model in use | A retirement date is coming (L6) |

Things worth a dashboard, without a page: drift, feedback trends, and cost per success.

## Why the obvious fixes fall short

| Obvious idea | Why it falls short |
|---|---|
| HTTP status dashboards | Bad answers return 200 |
| Log everything | Costly, and a privacy problem |
| 1% head sampling | Kept 21 of 1,973 errors |
| Mean latency | Described nobody |
| Sum `input_tokens` | Misses cached tokens |
| Add cache fields to `gen_ai.usage.input_tokens` | Counts them twice |
| Alert only on errors | Our hook's run had none that mattered |

## How Anthropic solved it

| Mechanism | What's published | Source |
|---|---|---|
| SDK and API tracing | OpenTelemetry export, in beta | API reference |
| Content modes | Metadata by default, full content by choice | API reference |
| Usage and cost reports | Admin API, by model, key and workspace | API docs |
| Console charts | Rate limit headroom and cache rate | Rate limits page |
| Claude Code | OpenTelemetry for teams (AI12) | Claude Code docs |
| Evals with monitoring | Production monitoring as a layer | Demystifying evals |

## The production architecture

This is a recommended reference design, not a specific company's system.

**Baseline:** application logs and an HTTP dashboard.

**Improved:** traces with tokens and versions. New problem: storage grows quickly, and prompts in traces are now a sensitive store.

**Production:** metadata for everything, content for a sample, and a pipeline that decides.

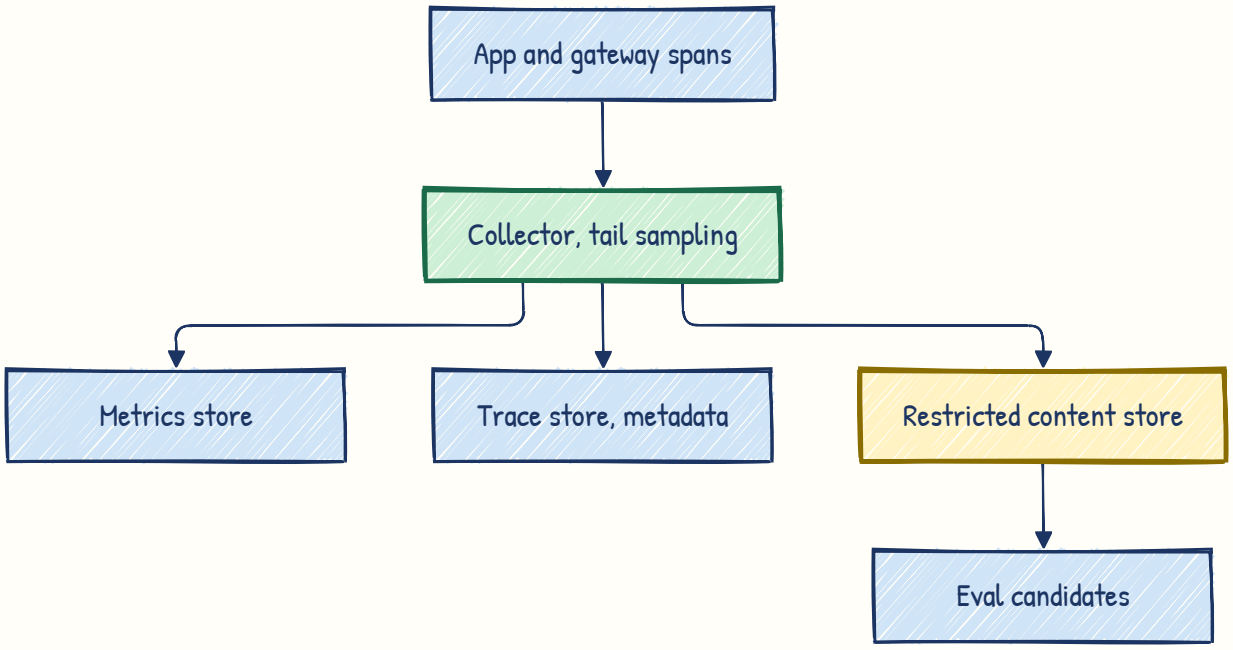



*Figure 3. All runs give metadata. A collector decides which traces to keep, once it knows how each ended.*

| Component | Why it exists | How it communicates |
|---|---|---|
| Instrumentation | Emits spans, with version and tenant attributes | OpenTelemetry |
| Collector | Sampling and redaction, in one place | OTLP |
| Metrics store | Dashboards and alerts | Aggregates from spans |
| Trace store | Debugging | Backend of your choice |
| Content store | Restricted and short-lived | Access control and audit |
| Reconciler | Compares totals with the provider's usage report | Daily job |
| Sampler to evals | Adds real failures to the suite (L3) | Review queue |

**Failures:**

| Failure | How you'd notice | What to do |
|---|---|---|
| Double-counted tokens | Reconciler disagrees with the invoice | Fix the sum, and document the fields |
| Metric cardinality explodes | Metrics bill and slow queries | No user IDs as labels. Keep those on traces |
| Trace content too large | Backend rejects spans | Content limits exist, per the reference, and earliest messages go first |
| Collector down | Missing data | Telemetry must never block requests |
| Beta attribute renamed | Dashboard goes blank | Pin the SDK, and alert on missing data |
| Prompts in general logs | A scan | AI09 L4 |

## Java

Separate the cost fields, and the totals follow:

```
long totalInput() {
    return input + cacheWrite + cacheRead;
}
```

Keeping these three separate is what makes hit ratio and cost both correct. A single "input" field can't be repaired later.

The sampling decision is one line each, and the only difference is what's known at that moment:

```
boolean head = random.nextDouble() < 0.01;
boolean tail = error || isSlow || random.nextDouble() < 0.001;
```

The second line can use `error` only because it runs after the outcome is known. In a real system, that's the collector.

## Trade-offs

| Decision | Benefit | Cost |
|---|---|---|
| Full content | Best debugging, and eval data | Privacy and storage |
| Metadata only | Safe and cheap | "Why did it say that?" stays open |
| Tail sampling | Keeps what matters | Collector state and memory |
| High-cardinality labels | Rich slicing | Cost. Use trace attributes instead |
| API tracing | No code change | Fewer platforms, and it's beta |
| Observability vendor | Quick start | Prompts leave your systems, so check that |

**When not to do this much:** a small internal tool can log `usage`, the stop reason and version IDs in one structured line per call. That already answers most questions.

### Common mistakes

**Building a cache hit ratio from `input_tokens` alone.** Use reads over total input, where total is the sum of all three fields.

### Common mistakes

**Putting a user ID in a metric label.** It multiplies your time series. Put it on the trace.

### What to say in an interview

"HTTP status tells me nothing about quality, so I trace each model turn and tool call, with tokens, stop reason, and prompt and model versions. I'm careful with token fields: the API's `input_tokens` excludes cache, while the OpenTelemetry attribute includes it. I reconcile with the provider's usage report. I use tail sampling to keep all errors, split latency by route, and treat stored prompts as sensitive."

## Certification lens

### How this lesson maps to CCAR-F

**Tracing tools aren't in the guide,** and the blueprint records that. The nearby ideas are keeping persistent case facts (TS 5.1), passing rich error context (TS 5.3), and measuring by segment (TS 5.5). Read `usage` fields correctly for real work, and AI03 L5 covers them on the API side.

These are original practice questions, not real or leaked.

**Q** Q1 · A cost dashboard shows spend falling by 90% after a release, while the invoice is unchanged. What's the likely cause?

**A** The release probably added or moved a cache breakpoint. `input_tokens` now excludes the cached part, and the dashboard sums only that field. Total input is `input_tokens` plus cache reads plus cache writes. Fix the sum, and reconcile with the provider's usage report daily.

**Q** Q2 · Why is tail sampling a better fit for agents than head sampling, and what does it cost?

**A** It decides after the outcome, so all errors and slow runs are kept, while head sampling at 1% keeps about 1% of them. The cost is a collector that buffers spans until each trace completes, and agent runs can last minutes, which increases memory and complexity.

**Q** Q3 · An agent's error rate is flat, yet complaints are rising. Name four signals you'd check.

**A** Any four: `max_tokens` stops (truncation), refusal rate, steps per run and limit endings (looping), tool error rate hidden inside successful runs, cache hit ratio, sampled eval scores by segment, drift in question types, and escalation rate.

## Revision - Lesson 4

→ A 200 says nothing about quality. Add traces, tokens, cost per outcome and quality.

→ Trace every model turn and tool call, with prompt, model and tool versions.

→ Messages API `input_tokens` excludes cache. Total is reads plus writes plus input.

→ `gen_ai.usage.input_tokens` already includes cache, so don't add to it. Sum one source only.

→ Reconcile with the provider's usage report.

→ Claude's OpenTelemetry export is in beta. It's metadata only by default.

→ Tail sampling keeps all errors. 1% head sampling kept 21 of 1,973.

→ Split latency by route, and report percentiles. First token and total are separate.

→ Drift means go and read samples. Alert on spend, cache ratio, and limit endings too.

*Your notes*